T6 Exercises – Fateme Firouznia

1. **What is the purpose of the /etc/passwd file, and how is it related to /etc/shadow?**

   On Linux systems, local user accounts are represented using a small set of text databases. The two most important ones are /etc/passwd and /etc/shadow, and they exist as a security split: general account information is kept readable for normal system operation, while password-related data is kept restricted.

   /etc/passwd is the primary local account database for identity information. It contains one line per account and stores fields such as the username, UID, primary GID, a comment/GECOS field, the user's home directory, and the login shell. This file is typically world-readable because many programs and services need to map UIDs to usernames and determine basic account properties. In my system, /etc/passwd is readable by everyone.

```
mohammad@template:~$ ls -l /etc/passwd /etc/shadow
-rw-r--r-- 1 root root   1789 Nov  6 15:22 /etc/passwd
-rw-r----- 1 root shadow  970 Nov  6 15:22 /etc/shadow
```

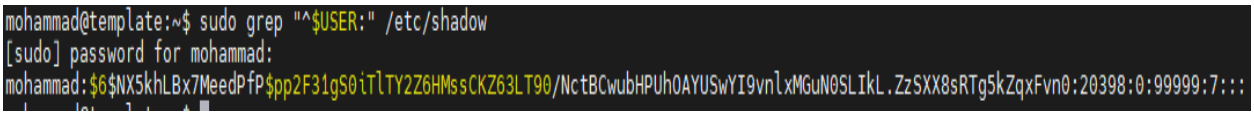
   Output showing that /etc/passwd is readable by all users, while /etc/shadow has restricted permissions

   In modern Linux distributions, /etc/passwd does **not** contain the password hash. Instead, the second field is commonly set to x (or a similar placeholder), which indicates that the actual password hash is stored elsewhere. You can see my user entry in /etc/passwd, including the x placeholder and the other account fields.

```
mohammad@template:~$ echo "$USER"
mohammad
mohammad@template:~$ grep "^$USER:" /etc/passwd
mohammad:x:1000:1000:mohammad:/home/mohammad:/bin/bash
```

   My user entry in /etc/passwd, showing the x placeholder and account fields.

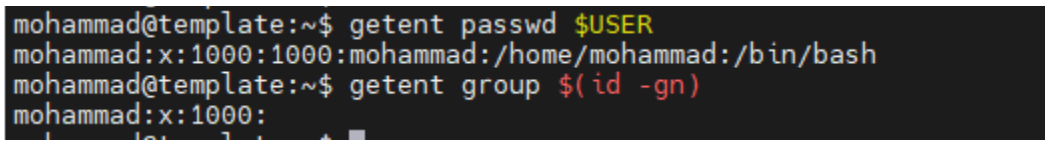
   /etc/shadow stores authentication-sensitive information, mainly the password hash and password aging/expiry metadata (for example: last password change date, minimum/maximum password age, warning period, and account expiry fields). Because this data can be abused for offline password cracking, /etc/shadow is protected with stricter permissions and is normally readable only by root (or by members of the shadow group, depending on the distribution). On my system, the permissions are clearly more restrictive than /etc/passwd.



My /etc/shadow entry with the password hash redacted, demonstrating restricted access.

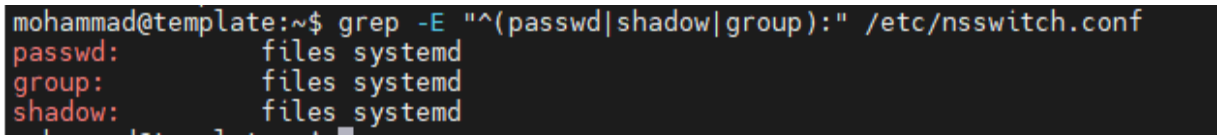
The relationship between the two files is straightforward: they are linked by **username**. The /etc/passwd entry provides identity and account settings (UID/GID/home/shell), while /etc/shadow provides the password hash and password policy for the same username. This is why a typical /etc/passwd record contains x in the password field: it points to the fact that the password hash is stored in /etc/shadow. My /etc/shadow entry is accessible only with elevated privileges, which matches the intended security design.

In practice, applications do not usually parse these files manually. For identity lookups, they use the system's standard name service interfaces (NSS), and a convenient way to demonstrate that path is getent. Running getent passwd <user> returns the account record as resolved by NSS, which commonly comes from /etc/passwd for local users. My getent output matches my /etc/passwd record, confirming the standard lookup behavior.



NSS-based lookup using getent, confirming standard account resolution.

The lookup order (for example, "files first, then other sources") is controlled by /etc/nsswitch.conf. On my system, the passwd, group, and shadow entries show that local files are part of the resolution method, which explains why /etc/passwd and /etc/shadow are consulted for local accounts.



Lookup order for passwd, group, and shadow as defined in /etc/nsswitch.conf.

For authentication (password verification and login policy), Linux commonly uses PAM (Pluggable Authentication Modules). PAM configuration is stored under /etc/pam.d/, and different services (login, sudo, sshd, etc.) can have different authentication stacks

Note: the exact module lines (e.g., pam_unix) can vary by distro and by service configuration, so it is normal that a quick grep may or may not show that string directly in a specific service file; many systems centralize rules in shared "common-*" PAM files.

```
mohammad@template:~$ ls -l /etc/pam.d/
total 88
-rw-r--r-- 1 root root  384 Feb 22  2024 chfn
-rw-r--r-- 1 root root   92 Feb 22  2024 chpasswd
-rw-r--r-- 1 root root  581 Feb 22  2024 chsh
-rw-r--r-- 1 root root 1208 Nov  6 15:41 common-account
-rw-r--r-- 1 root root 1242 Nov  6 15:41 common-auth
-rw-r--r-- 1 root root 1620 Nov  6 15:41 common-password
-rw-r--r-- 1 root root 1427 Nov  6 15:41 common-session
-rw-r--r-- 1 root root 1435 Nov  6 15:41 common-session-noninteractive
-rw-r--r-- 1 root root  606 Aug  5 17:14 cron
-rw-r--r-- 1 root root 4118 Apr  9  2024 login
-rw-r--r-- 1 root root   92 Feb 22  2024 newusers
-rw-r--r-- 1 root root  520 Oct 13  2022 other
-rw-r--r-- 1 root root   92 Feb 22  2024 passwd
-rw-r--r-- 1 root root  143 Mar  3  2024 runuser
-rw-r--r-- 1 root root  138 Mar  3  2024 runuser-l
-rw-r--r-- 1 root root 2133 Jun  9  2025 sshd
-rw-r--r-- 1 root root 2259 Mar  3  2024 su
-rw-r--r-- 1 root root  330 Jan 29  2024 sudo
-rw-r--r-- 1 root root  315 Jan 29  2024 sudo-i
-rw-r--r-- 1 root root  137 Mar  3  2024 su-l
-rw-r--r-- 1 root root  119 Aug  5 17:14 vmtoolsd
mohammad@template:~$ grep -n "pam_unix" /etc/pam.d/login 2>/dev/null || echo "no pam_unix line in /etc/pam.d/login"
8:# to disable any delay, you should add the nodelay option to pam_unix)
mohammad@template:~$ grep -n "pam_unix" /etc/pam.d/sshd 2>/dev/null || echo "no pam_unix line in /etc/pam.d/sshd"
no pam_unix line in /etc/pam.d/sshd
```

PAM configuration directory showing where authentication rules are defined.

Overall, /etc/passwd provides general account identity information and is readable for normal system functionality, while /etc/shadow stores password hashes and password policy data and is protected to reduce the risk of credential attacks. Together, they implement a standard and secure approach to managing local users on Linux.

2. **How can I enforce a password change policy for users every 100 days?**
   This question is about **password aging policy**: forcing users to change their passwords after a fixed period (here: 100 days). On Linux, this policy is stored per user (in the shadow password database) and can also be set as a system default for accounts created in the future. For an **existing user**, the standard tool is **`chage`** (change password age). First, you can inspect the current settings using **`chage -l`**. In my case, the maximum password age was effectively unlimited (99999 days).

```
mohammad@template:~$ sudo chage -l $USER
[sudo] password for mohammad:
Last password change                                    : Nov 06, 2025
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
mohammad@template:~$
```

Current password-aging settings for my account (before applying the 100-day policy).

To enforce a 100-day maximum password age, set the maximum age to 100 days. It is also common to set a warning period so the user is notified before the password expires (for example, 7 days): `**chage -M 100 -W 7 username**`. After applying the change, re-check with `**chage -l**` to confirm the new policy. You can see that the maximum number of days changed to 100 and that an explicit expiry date is now calculated.

```
mohammad@template:~$ sudo chage -M 100 -W 7 $USER
mohammad@template:~$ sudo chage -l $USER
Last password change                                    : Nov 06, 2025
Password expires                                        : Feb 14, 2026
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 100
Number of days of warning before password expires       : 7
```

Applying `chage -M 100 -W 7` and verifying the result with `chage -l` (maximum age becomes 100 days).

If you need to apply the same rule to multiple existing regular (non-system) users, you can run `**chage -M 100 -W 7**` for each account (commonly filtering for UID ≥ 1000 to avoid system accounts).

For **new users created in the future**, set the default aging parameters in `**/etc/login.defs**`. The key fields are `**PASS_MAX_DAYS**` (set to 100), along with optional defaults like `**PASS_MIN_DAYS**` and `**PASS_WARN_AGE**`. Note that changing `**/etc/login.defs**` affects newly created accounts; existing accounts still need to be updated with `**chage**`.

```
mohammad@template:~$ grep -E "^(PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE)" /etc/login.defs
PASS_MAX_DAYS   100
PASS_MIN_DAYS   1
PASS_WARN_AGE   7
mohammad@template:~$
```

Default password-aging settings for newly created users as defined in /etc/login.defs.

In summary, enforce the 100-day policy for existing accounts using `**chage -M 100**` (and optionally `**-W 7**`), verify with `**chage -l**`, and configure **/etc/login.defs** to ensure the same defaults apply to new accounts.

3. **Create a cron job that collects all system log files (/var/log/syslog) every day at 3:00 AM and archives them weekly on Friday night. Then move the weekly archive to another location.**
   A cron-based workflow was implemented to automate system log management. The solution consists of two scripts: a daily collector that copies /var/log/syslog every day at 03:00, and a weekly archiver that compresses the collected logs on Friday night and moves the archive to a backup directory. Directory structure and permissions created for log storage:

```
mohammad@template:~$ sudo mkdir -p /var/log/syslog-archive/daily
mohammad@template:~$ sudo mkdir -p /var/backups/syslog-weekly
mohammad@template:~$ sudo chmod 750 /var/log/syslog-archive /var/log/syslog-archive/daily /var/backups/syslog-weekly
mohammad@template:~$ sudo ls -ld /var/log/syslog-archive /var/log/syslog-archive/daily /var/backups/syslog-weekly
drwxr-x--- 2 root root 4096 Dec 27 15:06 /var/backups/syslog-weekly
drwxr-x--- 3 root root 4096 Dec 27 15:05 /var/log/syslog-archive
drwxr-x--- 2 root root 4096 Dec 27 15:05 /var/log/syslog-archive/daily
```

   The required directory structure was created to store daily logs and weekly archives. All directories are owned by root and configured with restricted permissions to prevent unauthorized access.

```
mohammad@template:~$ sudo tee /usr/local/sbin/collect_syslog_daily.sh > /dev/null <<'EOF'
#!/usr/bin/env bash
set -euo pipefail

SRC="/var/log/syslog"
BASE="/var/log/syslog-archive/daily"
WEEK_DIR="$BASE/$(date +%G-W%V)"
OUT="$WEEK_DIR/syslog-$(date +%F)"

mkdir -p "$WEEK_DIR"
cp -a "$SRC" "$OUT"
EOF
[sudo] password for mohammad:
mohammad@template:~$ sudo chmod 750 /usr/local/sbin/collect_syslog_daily.sh
mohammad@template:~$ sudo ls -l /usr/local/sbin/collect_syslog_daily.sh
-rwxr-x--- 1 root root 208 Dec 27 15:30 /usr/local/sbin/collect_syslog_daily.sh
```

   The daily script copies the system log file (/var/log/syslog) every day at 03:00 into a date-stamped file. The collected logs are organized by ISO week number to maintain a clear weekly structure.

```
mohammad@template:~$ sudo /usr/local/sbin/collect_syslog_daily.sh
mohammad@template:~$ sudo ls -l /var/log/syslog-archive/daily
total 4
drwxr-xr-x 2 root root 4096 Dec 27 15:31 2025-W52
mohammad@template:~$ sudo ls -l /var/log/syslog-archive/daily/$(date +%G-W%V)
total 300
-rw-r----- 1 syslog adm 303335 Dec 27 15:30 syslog-2025-12-27
```

To handle long-term storage, a second script performs weekly processing. On Friday night, all daily logs collected during the current week are compressed into a single archive and moved to a dedicated backup directory.

```
mohammad@template:~$ sudo /usr/local/sbin/archive_syslog_weekly.sh
mohammad@template:~$ sudo ls -l /var/backups/syslog-weekly
total 48
-rw-r--r-- 1 root root 46942 Dec 27 15:35 syslog-2025-W52.tar.gz
```

The contents of the weekly archive were verified to confirm that all expected daily log files were included.

```
mohammad@template:~$ sudo tar -tzf /var/backups/syslog-weekly/syslog-$(date +%G-W%V).tar.gz | head
2025-W52/
2025-W52/syslog-2025-12-27
```

Finally, cron jobs were configured in the root crontab to automate the daily and weekly execution of the scripts using absolute paths.

```
mohammad@template:~$ sudo crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0 3 * * * /usr/local/sbin/collect_syslog_daily.sh >> /var/log/syslog-archive/cron.log 2>&1
59 23 * * 5 /usr/local/sbin/archive_syslog_weekly.sh >> /var/log/syslog-archive/cron.log 2>&1
```

4. **Recreate question 3 using systemd-run instead of a traditional cron job.**
   In this question, the workflow implemented in Question 3 using cron jobs is recreated using systemd-run. Instead of static cron entries, transient systemd timer units are used to schedule the same daily and weekly log management tasks. systemd-run allows creating transient service and timer units directly from the command line without writing permanent unit files under /etc/systemd/system. These units are managed by systemd like regular services and fully integrated with system logging. This approach avoids maintaining permanent unit files while still leveraging systemd's scheduling and logging features.

   Script verification
   The existence and executability of the daily and weekly scripts were verified.

```
mohammad@template:~$ sudo ls -l /usr/local/sbin/collect_syslog_daily.sh /usr/local/sbin/archive_syslog_weekly.sh
[sudo] password for mohammad:
-rwxr-x--- 1 root root 515 Dec 27 15:33 /usr/local/sbin/archive_syslog_weekly.sh
-rwxr-x--- 1 root root 208 Dec 27 15:30 /usr/local/sbin/collect_syslog_daily.sh
```

   Daily timer creation
   The daily log collection task was scheduled to run every day at 03:00 using systemd-run and an OnCalendar expression. Persistent timers ensure reliability across reboots.

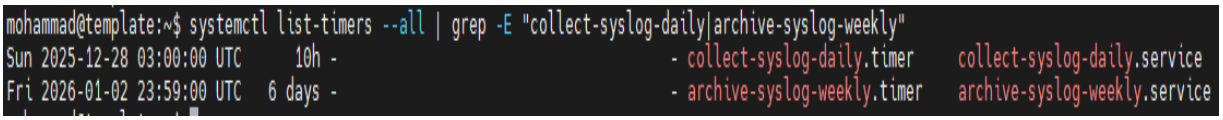
```
mohammad@template:~$ sudo systemd-run \
  --unit=collect-syslog-daily \
  --on-calendar="*-*-* 03:00:00" \
  --timer-property=Persistent=true \
  /usr/local/sbin/collect_syslog_daily.sh
Running timer as unit: collect-syslog-daily.timer
Will run service as unit: collect-syslog-daily.service
```

   Weekly timer creation
   The weekly archive task was scheduled to run on Friday night at 23:59 using a separate transient systemd timer.

```
mohammad@template:~$ sudo systemd-run \
  --unit=archive-syslog-weekly \
  --on-calendar="Fri *-*-* 23:59:00" \
  --timer-property=Persistent=true \
  /usr/local/sbin/archive_syslog_weekly.sh
Running timer as unit: archive-syslog-weekly.timer
Will run service as unit: archive-syslog-weekly.service
```

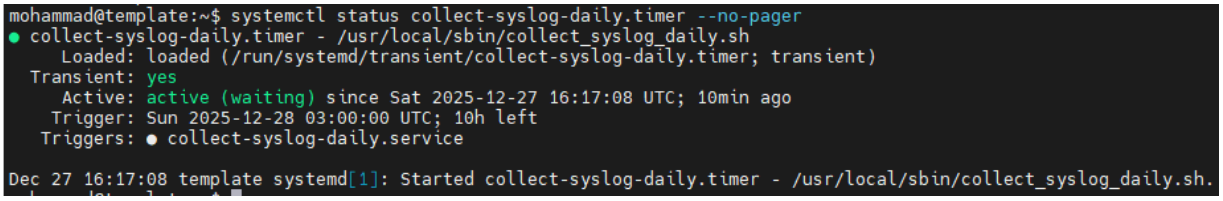
Timer verification

The list of active timers confirms that both daily and weekly timers were created successfully.



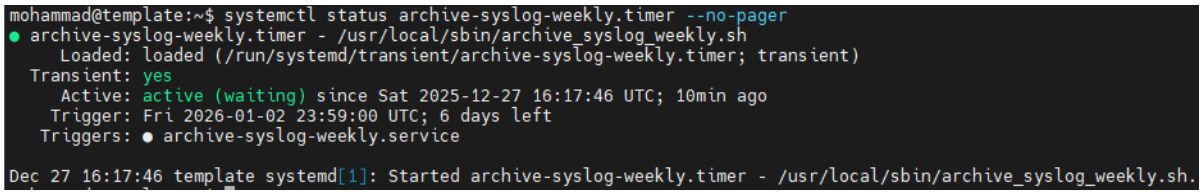
Daily timer status

The status output shows that the daily timer is active and waiting for its next trigger.



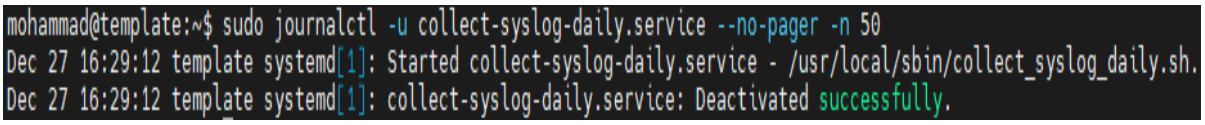
Weekly timer status

The weekly timer is active and correctly scheduled.



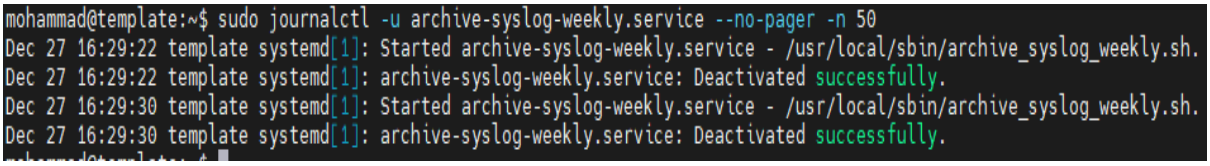
Execution logs (daily)

journalctl confirms that the daily service executed successfully.



Execution logs (weekly)

journalctl confirms that the weekly archive service executed successfully



Conclusion

This demonstrates that the cron-based workflow from Question 3 was successfully recreated using systemd-run, while benefiting from systemd's integrated logging, monitoring, and reliability features.
The behavior was verified on a running system.

5. **Use Chrony to set up a time synchronization service where vm1 acts as the Chrony server and vm2 synchronizes its time using vm1 as the source (do not use external time servers).**

This question explains how the hardware clock (RTC) works in Linux systems and what happens to the hardware clock when the device is powered off.

Conceptual background

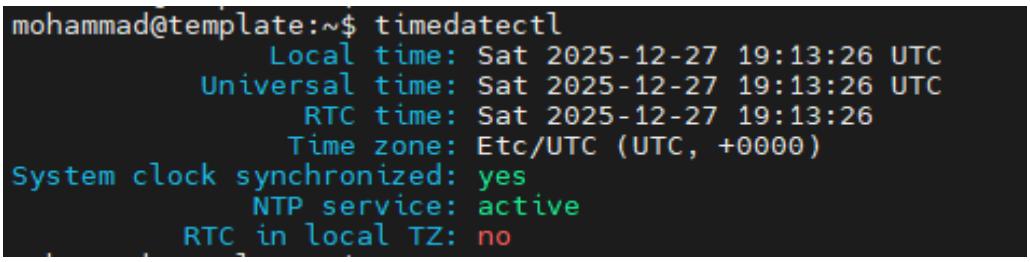
Linux systems maintain two clocks: the hardware clock (RTC) and the system clock. The hardware clock is a physical clock located on the motherboard and powered by a CMOS battery, which allows it to continue running even when the system is powered off. The system clock, in contrast, is maintained by the Linux kernel and exists only while the system is running. The hwclock utility acts as a bridge between these two clocks. During system startup, the kernel reads the current time from the hardware clock and initializes the system clock. The hwclock command can also be used to synchronize the system clock and the hardware clock.

Behavior during power-off

When the system is powered off, the system clock stops completely because it resides in memory. However, the hardware clock continues to run independently using its dedicated battery, preserving the correct time until the system is powered on again.

Verification on the system

On this virtualized system, direct access to the hardware clock using the hwclock command is not available. This behavior is expected in many virtual machines where the RTC is not directly exposed by the hypervisor. Therefore, the system time and RTC status were inspected using the timedatectl command.



Conclusion

This demonstrates that while the hardware clock is responsible for preserving time across power cycles, the system clock is initialized and synchronized at runtime. In virtualized environments, timekeeping is handled by the kernel and synchronization services such as NTP.

6. **Use Chrony to set up a time synchronization service where vm1 acts as the Chrony server and vm2 synchronizes its time using vm1 as the source (do not use external time servers).**

In this scenario, Chrony is used to configure an internal time synchronization service between two virtual machines. One machine (vm1) acts as the Chrony server, while a second machine (vm2) synchronizes its system time using vm1 as the only time source. External NTP servers are not used.
On vm1, Chrony is configured as a local time server by disabling all external NTP pools and enabling local time serving. The following configuration is applied:

```
# /etc/chrony/chrony.conf (vm1)
local stratum 10
allow 192.168.0.0/24
```

In this setup, vm1 acts as an internal reference clock. Although it does not synchronize with external NTP servers, it still provides a consistent time source within the local network, which is sufficient for internal synchronization scenarios.

This configuration allows vm1 to provide time to clients on the local network even in the absence of external time sources.
On vm2, all external NTP servers are removed, and vm1 is defined explicitly as the only time source:

```
# /etc/chrony/chrony.conf (vm2)
server <vm1-ip-address> iburst
```

After restarting the Chrony service on both systems, vm2 synchronizes its system clock with vm1. If implemented in practice, the synchronization status would be verified using chronyc sources and chronyc tracking on vm2.
The synchronization status can be verified using the following commands on vm2:

```
chronyc sources -v
chronyc tracking
```

These commands confirm that vm1 is selected as the active time source and that the system clock is synchronized.
In this environment, the configuration is described conceptually due to the availability of a single virtual machine. However, the same setup can be applied directly in practice by deploying a second virtual machine or using a dedicated internal time server.

7. **Explain in detail how the system clock works. What does it mean to synchronize the system time using NTP? What happens if the NTP server goes down? What is meant by 'time drift' in the context of NTP?**
   The system clock in Linux is maintained by the kernel and is initialized during system startup. It is based on hardware oscillators, which are not perfectly accurate and may drift over time due to hardware characteristics, temperature variations, and system load.
   Synchronizing the system time using NTP means continuously comparing the local system clock with one or more accurate reference time sources. An NTP client estimates the offset and drift of the local clock relative to these references and gradually adjusts the system clock to keep it aligned. Modern implementations such as Chrony avoid abrupt time jumps during normal operation and instead apply smooth corrections to maintain system stability.
   If the NTP server becomes unavailable, the system clock does not immediately fail. Instead, the NTP client continues to run the system clock using the last known drift estimation. Over time, however, small inaccuracies accumulate, and the system clock gradually deviates from the correct time until synchronization is restored.
   Time drift refers to the gradual divergence of the system clock from the true time due to the inherent inaccuracy of hardware oscillators. In the context of NTP, drift is continuously measured and compensated for, allowing the system clock to remain accurate even in environments where network conditions or time sources are unstable.

8. **Write a script that logs user login attempts when a wrong password is entered. This script should be implemented as a systemd service unit.**
   To log user login attempts when an incorrect password is entered, a script was created to monitor the system authentication logs and detect failed authentication events.

On Linux systems, failed login attempts are recorded in /var/log/auth.log. The script continuously monitors this log file and searches for authentication failure messages such as “Failed password” or “authentication failure”. Whenever such an event is detected, the script appends the corresponding log entry to a dedicated log file together with a timestamp.
The script is implemented as a systemd service unit to ensure continuous operation and automatic startup after system reboots. The service runs in the background and is configured to restart automatically if it stops unexpectedly, providing persistent monitoring without manual intervention.

Script implementation

```
#!/bin/bash

LOG_SOURCE="/var/log/auth.log"
LOG_TARGET="/var/log/failed_login_attempts.log"

tail -Fn0 "$LOG_SOURCE" | while read line; do
    if echo "$line" | grep -E "Failed password|authentication failure"; then
        echo "$(date '+%Y-%m-%d %H:%M:%S') - $line" >> "$LOG_TARGET"
    fi
done
```

The script is executed by systemd using the following service unit definition.

systemd service unit

```
[Unit]
Description=Log failed user login attempts
After=network.target

[Service]
ExecStart=/usr/local/bin/failed_login_logger.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

The service is enabled to start automatically at boot time using systemd’s multi-user target. This implementation demonstrates how system authentication logs can be processed in real time using a custom script and how systemd services can be used to manage long-running background monitoring tasks reliably.

9. **Set up an rsyslog server on vm1 and configure it to receive and store logs from /var/log, journalctl, and dmesg from other systems.**

   In this setup, rsyslog is configured to act as a centralized logging server on vm1. The server receives log messages over the network from other systems and stores them locally for analysis and auditing.This setup represents a typical centralized logging architecture used in multi-system Linux environments.

   On vm1, rsyslog is configured to listen for incoming syslog messages using both UDP and TCP. The TCP and UDP input modules are enabled, and logs are stored in a structured directory hierarchy based on the hostname of the sending system. This allows logs from multiple systems to be organized and inspected separately.

   Client systems are configured to forward all syslog messages to the central rsyslog server. Rsyslog is able to read messages from systemd's journal and kernel logging interfaces, which allows logs generated by /var/log, journalctl, and dmesg to be forwarded without additional configuration. Since these logs are processed by rsyslog, they are automatically transmitted to the server.

   This configuration enables centralized collection of authentication logs, system logs, and kernel messages from multiple machines, providing a unified and scalable logging solution.